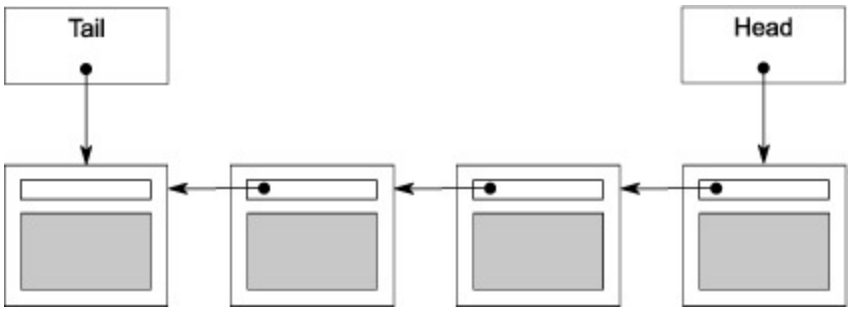
```
        std::shared_ptr<T> const res(
            std::make_shared<T>(std::move(head->data)));
        std::unique_ptr<node> const old_head=std::move(head);
        head=std::move(old_head->next);                    <-- 3
        return res;
    }

    void push(T new_value)
    {
        std::unique_ptr<node> p(new node(std::move(new_value)));
        node* const new_tail=p.get();
        if(tail)
        {
            tail->next=std::move(p);                       <-- 4
        }
        else
        {
            head=std::move(p);                             <-- 5
        }
        tail=new_tail;                                     <-- 6
    }
};
```

Figure 6.1. A queue represented using a single-linked list



First off, note that listing 6.4 uses `std::unique_ptr<node>` to manage the nodes, because this ensures that they (and the data they refer to) get deleted when they're no longer needed, without having to write an explicit `delete`. This ownership chain is managed from `head`, with `tail` being a raw pointer to the last node.

Although this implementation works fine in a single-threaded context, a couple of things will cause you problems if you try to use fine-grained locking in a multithreaded context. Given that you have two data items